



Lab02

REPORT

Applied Machine Learning - COS30082

Student Name: Truong Ngoc Gia Hieu

Student ID (VietNam): SWS01217

StudentID (Australia): 105565520

A. Logistic Regression Model

In the second laboratory of the unit, the task requires students to utilize two models including Logistic Regression and SVM Model to predict Titanic survival. In the first section of the document, I use the Logistic Regression Model to predict whether passengers survived or not based on their personal attributes such as age, gender, and ticket class.

First and foremost, I need to load the dataset named *Titanic-Dataset.csv* and print first five rows of the dataset.

```
#-- Import Library. --#
import pandas as pd
#-- Import Dataset. --#
titanic_dataset = pd.read_csv('Titanic-Dataset.csv')
#-- Print First 5 Rows. --#
titanic_dataset.head(5)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Figure 1: Overview of the Titanic Raw Dataset.

Next, I use the following method to showcase crucial information:

- *.info()*: Displays column names, non-null value counts, and the data types of each column.
- *.describe()*: Showcases statistical summary of the dataset.
- *.isnull().sum()*: Counts the total number of missing (null) values for each column.

```
#-- Information of the dataset. --#
titanic_dataset.info()
```

<class 'pandas.core.frame.DataFrame'>			
RangeIndex: 891 entries, 0 to 890			
Data columns (total 12 columns):			
#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Sex	891 non-null	object
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Cabin	204 non-null	object
11	Embarked	889 non-null	object
dtypes: float64(2), int64(5), object(5)			
memory usage: 83.7+ KB			

Figure 2: Dataset Structure and Data Types Overview

```
#-- Statistical of the Dataset. --#
titanic_dataset.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Figure 3: Statistical Summary of the Dataset

```
titanic_dataset.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Figure 4: Missing Values Count per Feature

```
print(f"Number of Rows: {titanic_dataset.shape[0]}")
print(f"Number of Columns: {titanic_dataset.shape[1]}")

Number of Rows: 891
Number of Columns: 12
```

Figure 5: Total Number of Rows and Columns

After checking the number of null values, there are about 177 null values in the feature *Age*, 2 values in the *Embarked* column and 687 values in the *Cabin* feature. From my perspective, I suggest using the median method in terms of filling missing age values, as it is robust against outliers and provides a stable central tendency. Meanwhile, because the *Cabin* column is the categorical data type so in this case, I decide to make a new group named *Unknown* group. Notably, the reason why I do not delete because number of missing value is large which is 687 out of 903 dataset.

```
titanic_dataset['Age'].fillna(titanic_dataset['Age'].median(), inplace=True)
titanic_dataset['Cabin'].fillna('Unknown', inplace=True)
titanic_dataset['Embarked'].fillna(titanic_dataset['Embarked'].mode()[0], inplace=True)

/tmp/ipykernel_2895/1715433734.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method({col: value}, inplace=True)

titanic_dataset['Age'].fillna(titanic_dataset['Age'].median(), inplace=True)
/tmp/ipykernel_2895/1715433734.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method({col: value}, inplace=True)

titanic_dataset['Embarked'].fillna(titanic_dataset['Embarked'].mode()[0], inplace=True)
```

Figure 6: Handling Missing Values for Age and Cabin Features

```

▶ #-- Check Missing Values After Dealing Missing Data. --#
titanic_dataset.isnull().sum()

...
0

PassengerId  0
Survived     0
Pclass       0
Name         0
Sex          0
Age         0
SibSp        0
Parch        0
Ticket       0
Fare         0
Cabin        0
Embarked     0

dtype: int64

```

Figure 7: Verification of Missing Values After Imputation

Subsequently, because both Logistic Regression and SVM models cannot understand text data such as gender categories and embarked ports, these categorical features must be encoded into numerical representations. In detail, I drop out column ***Ticket*** because it is not necessary for prediction, encode column ***Sex*** (0 for male and 1 for female), and One-Hot Encoding technique for ***Embarked*** and ***Cabin*** feature.

```

#-- Encoding Sex/Gender Feature into 0 (Male) and 1 (Female). --#
titanic_dataset['Sex'] = titanic_dataset['Sex'].map({'male':0, 'female':1})
#-- Drop the Ticket Column. --#
titanic_dataset.drop('Ticket', axis=1, inplace=True)
#-- Apply One-Hot Encoding for the Embarked and Cabin column. --#
titanic_dataset = pd.get_dummies(titanic_dataset, columns=['Embarked', 'Cabin'], drop_first=True)

```

Figure 8: Categorical Encoding for Categorical Features

Based on the requirement of the lab, the target variable was defined as ***Survived*** while the remaining columns were used as input features for prediction.

Step 5 - Features and Target

```
x = df.drop("Survived", axis=1)
y = df["Survived"]

print(x.shape)
print(y.shape)

(891, 8)
(891,)
```

Figure 9: Separation of Features and Target Variable

In addition, the dataset was then divided into training and testing sets using an 80:20 ratio. The training set was used to train the models while the testing set was reserved for evaluating their performance on unseen data.

Step 6 - Train Test Split

```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(x_train.shape)
print(x_test.shape)

(712, 8)
(179, 8)
```

Figure 10: Train-Test Split and Data Dimensionality

Therefore, for ensuring reproducibility of the experiment, a fixed random state was applied during the splitting process. Furthermore, feature scaling was performed using the StandardScaler technique.

Step 7 - Feature Scaling

```
scaler = StandardScaler()

x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Figure 11: Feature Scaling using StandardScaler

After import dataset, exploratory data, choose feature and split into train and test part, I move to training dataset with Logistic Regression and SVM Model.

Step 8 - Logistic Regression

```
log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train_scaled, y_train)

log_pred = log_model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, log_pred))
print("Precision:", precision_score(y_test, log_pred))
print("Recall:", recall_score(y_test, log_pred))
print("F1 Score:", f1_score(y_test, log_pred))

print(classification_report(y_test, log_pred))
```

```
Accuracy: 0.8044692737430168
Precision: 0.7931034482758621
Recall: 0.6666666666666666
F1 Score: 0.7244094488188977
```

	precision	recall	f1-score	support
0	0.81	0.89	0.85	110
1	0.79	0.67	0.72	69
accuracy			0.80	179
macro avg	0.80	0.78	0.79	179
weighted avg	0.80	0.80	0.80	179

Figure 12: Logistic Regression Model Training and Evaluation Metrics

Step 9 - SVM

```
svm_model = SVC(kernel="rbf")

svm_model.fit(X_train_scaled, y_train)

svm_pred = svm_model.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, svm_pred))
print("Precision:", precision_score(y_test, svm_pred))
print("Recall:", recall_score(y_test, svm_pred))
print("F1 Score:", f1_score(y_test, svm_pred))

print(classification_report(y_test, svm_pred))
```

*** Accuracy: 0.8100558659217877
Precision: 0.8431372549019608
Recall: 0.6231884057971014
F1 Score: 0.7166666666666667

	precision	recall	f1-score	support
0	0.80	0.93	0.86	110
1	0.84	0.62	0.72	69
accuracy			0.81	179
macro avg	0.82	0.78	0.79	179
weighted avg	0.81	0.81	0.80	179

Figure 13: Support Vector Machine (SVM) Model Training and Evaluation Metrics

After that, I compare accuracy between Logistic Regression and SVM.

Step 10 - Final Comparison

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "SVM"],
    "Accuracy": [
        accuracy_score(y_test, log_pred),
        accuracy_score(y_test, svm_pred)
    ]
})

results.sort_values(by="Accuracy", ascending=False)
```

	Model	Accuracy
1	SVM	0.810056
0	Logistic Regression	0.804469

Figure 14: Final Model Performance Comparison (Logistic Regression and SVM)